

Documentação dos Eventos — MOM (Sprint 2)

Projeto: BarberTime — agendamento em barbearia
Disciplina: LDAMD — PUC Minas
Sprint: 2 — Integração com MOM
Broker: RabbitMQ 3.13 (exchange topic)

Visão geral

O backend REST **publica** eventos após persistir no PostgreSQL. Um **worker separado** consome as filas e simula a notificação (log), **sem chamada REST** entre produtor e consumidor.

Topologia

Componente	Nome	Tipo
Exchange	barbertime.events	topic (durable)
Fila (barbeiro)	barbertime.notifications.barber	durable
Fila (cliente)	barbertime.notifications.client	durable
Routing key	appointment.requested	→ fila do barbeiro
Routing key	appointment.confirmed	→ fila do cliente

Envelope comum

Todas as mensagens utilizam o mesmo formato:

```
{
  "eventType": "<tipo>",
  "eventId": "<uuid>",
  "timestamp": "<ISO-8601>",
  "source": "barbertime-api",
  "payload": { }
```

Evento 1 — Nova solicitação de agendamento

Campo	Valor
Nome	appointment.requested

Momento	Após POST /api/v1/appointments (status pending no banco)
Produtor	API REST — <code>EventPublisher.publishAppointmentRequested()</code>
Consumidor	Worker — fila <code>barbertime.notifications.barber</code>
Destinatário final	Barbeiro (app prestador / e-mail)
Exchange / routing key	<code>barbertime.events / appointment.requested</code>

Payload de exemplo:

```
{
  "eventType": "appointment.requested",
  "eventId": "e44bb586-b6fd-4f3c-ada0-d1c53e511f21",
  "timestamp": "2026-05-25T16:40:00.000Z",
  "source": "barbertime-api",
  "payload": {
    "appointmentId": "24f21de9-f6f5-428e-b45a-ccf50eb2b364",
    "barberId": "11111111-1111-1111-1111-111111111111",
    "barberName": "João Barbeiro",
    "barberEmail": "joao.barbeiro@barbertime.seed",
    "clientFullName": "Caio Kodato Cliente",
    "startsAt": "2026-06-27T17:30:00.000Z",
    "endsAt": "2026-06-27T18:00:00.000Z"
  }
}
```

Evento 2 — Agendamento confirmado pelo barbeiro

Campo	Valor
Nome	<code>appointment.confirmed</code>
Momento	Após PATCH /api/v1/barber/appointments/:id/confirm (status confirmed)
Produtor	API REST — <code>EventPublisher.publishAppointmentConfirmed()</code>
Consumidor	Worker — fila <code>barbertime.notifications.client</code>
Destinatário final	Cliente (app cliente / e-mail)
Exchange / routing key	<code>barbertime.events / appointment.confirmed</code>

Payload de exemplo:

```
{
  "eventType": "appointment.confirmed",
  "eventId": "7c10eec4-b808-42e3-acdb-b3ca828a452f",
  "timestamp": "2026-05-25T16:55:00.000Z",
  "source": "barbertime-api",
  "payload": {
    "appointmentId": "24f21de9-f6f5-428e-b45a-ccf50eb2b364",
    "clientFullName": "Caio Kodato Cliente",
    "clientEmail": "caiovictorkodatot@gmail.com",
    "barberName": "João Barbeiro",
    "startsAt": "2026-06-27T17:30:00.000Z",
    "endsAt": "2026-06-27T18:00:00.000Z"
  }
}
```

Fluxo assíncrono

Etapa	Ação
1	Cliente: <code>POST /appointments</code> → API persiste <code>pending</code> → publica <code>appointment.requested</code>
2	Worker consome fila do barbeiro → log de notificação ao barbeiro
3	Barbeiro: <code>PATCH .../confirm</code> → API atualiza <code>confirmed</code> → publica <code>appointment.confirmed</code>
4	Worker consome fila do cliente → log de notificação ao cliente

Evidências: logs do worker; painel RabbitMQ em <http://localhost:15673> (`barbertime` / `barbertime`).